

Load Testing Documentation

This document describes the load testing methodology for comparing the REST and GraphQL implementations of the Apartment Booking System backend. It defines the test environment, scenario design, metrics, execution process, and reporting approach used for formal benchmark analysis.

1. Overview

The load testing suite compares REST and GraphQL under equivalent operating conditions. Both APIs use the same database, service layer, validation rules, authentication rules, and business logic. The primary difference under test is the API architecture and request shape.

The benchmark is designed to answer the following questions:

- How do REST and GraphQL compare in latency under equivalent load?
- How do they compare in throughput?
- How much CPU and memory does each API consume?
- How does response payload size differ by scenario?
- How do the architectures behave for relational reads, nested reads, and writes?

The suite uses k6 for traffic generation, Docker resource statistics for container-level resource monitoring, and a Node.js comparison script for result aggregation.

2. Authentication Requirement

The k6 scenarios authenticate by calling login with a phone number and using the returned JWT token:

```
{ "phoneNumber": "+6281234567892" }
```

For this to work, OTP must be disabled during load tests:

```
DISABLE_OTP=true
```

When `DISABLE_OTP=true`, the REST and GraphQL login endpoints return a token directly. When it is not enabled, login returns an OTP session, which is correct for normal OTP flow but incompatible with the current k6 authentication helper.

Before running any authenticated scenario, verify both API containers have the setting:

```
docker exec aps-rest-api printenv DISABLE_OTP
docker exec aps-graphql-api printenv DISABLE_OTP
```

Both commands should print:

```
true
```

If `.env` is changed, recreate the API containers:

```
docker-compose up -d --force-recreate rest-api graphql-api
```

3. Test Environment

Required Services

The benchmark expects these services to be available:

Service	Default URL	Purpose
REST API	<code>http://localhost:3001/api</code>	REST benchmark target
GraphQL API	<code>http://localhost:3002/graphql</code>	GraphQL benchmark target
PostgreSQL	configured through <code>.env</code>	Shared data store

Required Tools

Tool	Purpose
Docker and Docker Compose	Run the backend services with consistent limits
k6	Generate load and collect HTTP metrics
Node.js	Run result comparison scripts
jq	Optional command-line JSON inspection

Key Environment Settings

The following settings should be recorded for each formal run:

Variable	Expected use
<code>DISABLE_OTP=true</code>	Enables direct token login for k6
<code>REST_PORT=3001</code>	REST API host port
<code>GRAPHQL_PORT=3002</code>	GraphQL API host port
<code>JWT_SECRET</code>	Must be consistent for token signing and verification
<code>JWT_EXPIRES_IN</code>	Token lifetime

Variable	Expected use
DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME	Database connection
LOG_LEVEL	Logging verbosity

For Docker Compose runs, `docker-compose.yml` passes `DISABLE_OTP`, JWT, database, logging, and OTP provider variables into both API services.

4. Experimental Controls

The comparison is meaningful only if the following controls remain stable:

- Both APIs run on the same host machine.
- Both APIs use the same database and seed data.
- Both APIs use the same Docker CPU and memory limits.
- Both APIs use the same load profile.
- Both APIs use the same authentication mode.
- No code changes are made between paired REST and GraphQL runs.
- The host machine is not under unrelated heavy workload.

The current Docker Compose configuration assigns equivalent resource limits to both API containers.

5. Load Profile

The shared k6 configuration is defined in:

```
tests/k6/config/config.js
```

The current read and write profiles use the same stage structure:

Stage	Duration	Target
Warm-up	1 minute	100 virtual users
Sustained load	3 minutes	100 virtual users
Cooldown	1 minute	0 virtual users

Default thresholds:

Metric	Threshold
http_req_duration	p50 < 500 ms, p95 < 1000 ms, p99 < 2000 ms
http_reqs	rate > 30 requests per second
http_req_failed	rate < 1 percent
checks	rate > 99 percent

Scenario 06 uses write-specific thresholds because booking creation can include expected conflicts.

6. Test Scenarios

Scenario 01: Simple List

Purpose: establish baseline read performance.

REST:

```
GET /api/facilities
```

GraphQL:

```
query GetFacilities {  
  facilities {  
    id  
    name  
    description  
    pricePerHour  
    openTime  
    closeTime  
    isActive  
  }  
}
```

This scenario is public and does not require authentication.

Scenario 02: List with Relations

Purpose: compare retrieval of related booking, user, and facility data.

REST:

```
GET /api/users/:id/bookings?limit=10&offset=0
```

GraphQL:

```
query GetUserBookings($userId: Int!, $limit: Int!, $offset: Int!) {  
  userBookings(userId: $userId, limit: $limit, offset: $offset) {  
    id  
    bookingDate  
    startTime  
    endTime  
    status  
    totalPrice  
  }  
}
```

```
    user { id fullName phoneNumber role }
    facility { id name pricePerHour openTime closeTime }
  }
}
```

This scenario requires a JWT token.

Scenario 03: Filtered Slots

Purpose: measure parameterized availability lookup and slot generation.

REST:

```
GET /api/facilities/:id/slots?date=YYYY-MM-DD
```

GraphQL:

```
query GetAvailableSlots($input: SlotAvailabilityInput!) {
  availableSlots(input: $input) {
    facilityId
    date
    slots {
      startTime
      endTime
      isAvailable
    }
  }
}
```

This scenario is public and does not require authentication.

Scenario 04: Generic Nested Query

Purpose: compare a generic composition strategy.

REST performs multiple requests and composes the result client-side:

```
GET /api/users/:id
GET /api/users/:id/bookings?limit=10&offset=0
```

GraphQL performs a single request with multiple fields:

```
query GetDashboardGeneric($userId: Int!, $limit: Int!, $offset: Int!) {
  user(id: $userId) {
    id
```

```
    fullName
    phoneNumber
    role
    isVerifiedTenant
  }
  userBookingsGeneric(userId: $userId, limit: $limit, offset: $offset) {
    id
    bookingDate
    startTime
    endTime
    status
    totalPrice
    facility { id name pricePerHour }
  }
}
```

This scenario requires a JWT token.

Scenario 05: Optimized Nested Query

Purpose: compare dedicated dashboard retrieval for both API styles.

REST:

```
GET /api/users/:id/dashboard?limit=10&offset=0
```

GraphQL:

```
query GetUserDashboard($userId: Int!, $limit: Int!, $offset: Int!) {
  userDashboard(userId: $userId, limit: $limit, offset: $offset) {
    user { id fullName phoneNumber role isVerifiedTenant }
    bookings {
      id
      bookingDate
      startTime
      endTime
      status
      totalPrice
      facility { id name pricePerHour }
      user { id fullName }
    }
    bookingCountToday
    upcomingBookings
    totalSpent
  }
}
```

This scenario requires a JWT token.

Scenario 06: Booking Creation

Purpose: measure write performance, validation, conflict behavior, and cleanup.

REST:

```
POST /api/bookings
DELETE /api/bookings/:id/hard
```

GraphQL:

```
mutation CreateBooking($input: CreateBookingInput!) {
  createBooking(input: $input) {
    id
    status
    totalPrice
  }
}
```

The scenario creates bookings using test users and deletes successful bookings with an admin token. Some booking conflicts can occur under concurrent load and should be analyzed separately from system failures.

7. Metrics Collected

k6 Metrics

Metric	Meaning
http_req_duration	End-to-end request duration
http_reqs	Request throughput and total count
http_req_failed	Failed HTTP request rate
checks	Scenario-specific validation pass rate
data_received	Response payload and protocol bytes received
data_sent	Request payload and protocol bytes sent

Custom Scenario Metrics

Metric	Used by
facility_count	Scenario 01
response_size_bytes	Multiple read scenarios
bookings_retrieved	Scenario 02

Metric	Used by
<code>nested_objects_count</code>	Scenario 02
<code>available_slots_count</code>	Scenario 03
<code>booked_slots_count</code>	Scenario 03
<code>requests_per_dashboard</code>	Scenario 04
<code>user_bookings_count</code>	Scenario 05
<code>booking_success</code>	Scenario 06
<code>booking_conflict</code>	Scenario 06
<code>bookings_created</code>	Scenario 06
<code>booking_deleted</code>	Scenario 06

Docker Resource Metrics

The resource monitor records:

- Timestamp.
- Container name.
- CPU percentage.
- Memory usage in MB.
- Memory limit in MB.
- Memory percentage.
- Network input.
- Network output.

Resource files are written as CSV.

8. Running Tests

Start Services

```
docker-compose up -d --build
docker-compose ps
```

Verify Authentication Mode

```
docker exec aps-rest-api printenv DISABLE_OTP
docker exec aps-graphql-api printenv DISABLE_OTP
```

Both should return `true`.

Run One Scenario

REST:

```
k6 run tests/k6/scenarios/01-simple-list-rest.js
```

GraphQL:

```
k6 run tests/k6/scenarios/01-simple-list-graphql.js
```

Run All Scenarios

```
./tests/k6/run-all.sh
```

The runner executes each REST and GraphQL scenario as a pair and writes summary, raw, and resource files under **tests/results/**.

Override Target URLs

```
REST_URL=http://localhost:3001/api \  
GRAPHQL_URL=http://localhost:3002/graphql \  
./tests/k6/run-all.sh
```

9. Resource Monitoring

The full runner starts resource monitoring automatically. To run it manually:

```
./scripts/monitor-resources.sh 300 tests/results/resources.csv all
```

To monitor only one API container:

```
./scripts/monitor-resources.sh 300 tests/results/rest-only.csv aps-rest-api  
./scripts/monitor-resources.sh 300 tests/results/graphql-only.csv aps-  
graphql-api
```

Manual Docker observation:

```
docker stats aps-rest-api aps-graphql-api
```

10. Result Files

The benchmark writes files in this structure:

```
tests/results/  
  rest/  
    01-simple-list-raw.json  
    01-simple-list-summary.json  
    01-simple-list-resources.csv  
    02-list-with-relations-raw.json  
    02-list-with-relations-summary.json  
    02-list-with-relations-resources.csv  
    ...  
  graphql/  
    01-simple-list-raw.json  
    01-simple-list-summary.json  
    01-simple-list-resources.csv  
    ...
```

Some individual k6 scripts also define `handleSummary` outputs without the `-summary` suffix. For formal comparison, use the `*-summary.json` files generated by `run-all.sh`.

11. Analysis

Run the comparison script:

```
node scripts/compare-results.js
```

The script compares each scenario across REST and GraphQL using:

- Average latency.
- Median latency.
- p90 latency.
- p95 latency.
- Requests per second.
- Total requests.
- Error rate.
- Average CPU usage.
- Average memory usage.
- Data received.
- Data sent.

For direct inspection:

```
jq '.metrics.http_req_duration' tests/results/rest/01-simple-list-summary.json
```

```
jq '.metrics.http_reqs' tests/results/graphql/01-simple-list-summary.json  
head tests/results/rest/01-simple-list-resources.csv
```

12. Research Methodology

Independent Variable

The independent variable is API architecture:

- REST.
- GraphQL.

Dependent Variables

The dependent variables are:

- Latency.
- Throughput.
- Error rate.
- CPU usage.
- Memory usage.
- Network transfer.
- Payload size.

Controlled Variables

The controlled variables are:

- Hardware.
- Docker resource limits.
- Database and seed data.
- Authentication mode.
- Business logic.
- Load profile.
- Test duration.
- Test users.

Recommended Repetition

For formal reporting, run the full scenario set at least three times. Use the median value for the main comparison table, or report each run separately with a clear explanation of variation.

13. Reporting Guidance

A formal report should include:

- Test environment description.
- API versions or git commit.
- Database seed description.
- Docker resource limits.

- Authentication setting, including `DISABLE_OTP=true`.
- Load profile.
- Scenario definitions.
- Result tables for latency, throughput, error rate, CPU, memory, and data transfer.
- Notes on booking conflicts in Scenario 06.
- Limitations and threats to validity.

Suggested table format:

Scenario	API	Avg latency	P95 latency	Requests/sec	Error rate	Avg CPU	Avg memory
01 Simple list	REST						
01 Simple list	GraphQL						

14. Troubleshooting

Authenticated scenarios fail

Confirm that `DISABLE_OTP=true` is active in both API containers. Also verify that the test users in `tests/k6/config/config.js` exist in the database.

Login returns an OTP session

The application is running with OTP enabled. Set `DISABLE_OTP=true`, recreate the API containers, and repeat the login check.

High error rate in Scenario 06

Booking conflicts are expected up to the configured threshold. Investigate if the failures are 500-level errors, authentication errors, validation errors, or conflict rates above the threshold.

Results are inconsistent

Check for background system load, database contention, container restarts, thermal throttling, or changes in seed data. Repeat the run and compare medians.

Resource CSV is empty

Confirm the container name passed to `monitor-resources.sh` matches the running Docker container:

```
docker ps --format "table {{.Names}}\t{{.Status}}"
```

15. File Reference

Path	Purpose
/	

Path	Purpose
tests/k6/config/config.js	Shared k6 endpoints, users, stages, thresholds
tests/k6/utils/auth.js	Login and authorization helpers
tests/k6/utils/data.js	Test data helpers
tests/k6/scenarios/	REST and GraphQL scenario scripts
tests/k6/run-all.sh	Paired benchmark runner
scripts/monitor-resources.sh	Docker resource collection
scripts/compare-results.js	REST vs GraphQL result comparison
tests/results/	Generated benchmark outputs

16. References

- k6 documentation: <https://k6.io/docs/>
- Docker stats documentation: <https://docs.docker.com/reference/cli/docker/container/stats/>
- GraphQL specification: <https://spec.graphql.org/>